Does a Domain-Specific BPE Tokenizer Improve In-Domain Language Modeling?

An Evaluation on Medical Text

Samih Amer
Individual project — GPT-from-Scratch course final project

May 5, 2026

1 Introduction

Tokenization is the first transformation applied to text before an LLM ever sees it, and it quietly constrains what the model can learn: any word that is not represented as a single token must be learned as a sequence of subwords, and any linguistic unit the tokenizer fails to respect becomes a unit the model has to reconstruct from pieces. Medical text is unusually dense with specialized terminology (`cholecystectomy`, `laparoscopic`, `pneumonia`, `hypertension`, `intraoperative`) that a general-purpose tokenizer trained on web text splits into 4–6 subwords each, even though each of these is a single, semantically unified clinical concept.

This project asks a straightforward question: **if we swap in a BPE tokenizer trained on medical text, while holding every other aspect of the model and training pipeline fixed, how does the model's held-out performance change on (a) in-domain medical text, (b) out-of-domain general text, and (c) cross-domain medical text?** I trained otherwise-identical 29M-parameter GPT models on the same 100k PubMed abstracts under three different tokenizers — a general BPE trained on wikitext-103, a medical BPE trained on the same PubMed corpus, and an MTSamples BPE trained on a corpus of clinical transcripts — and evaluated all three against three held-out corpora using bits-per-character (BPC), the only cross-tokenizer-comparable metric. To distinguish real effects from seed noise, the GPT training was repeated over multiple seeds (5 / 5 / 3 for general / medical / mtsamples) and inferential statistics (paired t-tests, 95% CIs, Cohen's d) were run on the seed-matched results.

Key results. The headline finding is a **register-distinctness scaling**: the size of a domain tokenizer's win is approximately a function of how distinct its training register is from general English. The MTSamples-trained tokenizer beats general by **0.78 BPC** on MTSamples eval (n=3, 95% CI [0.73, 0.83], p=0.0002) and beats medical by 1.28 BPC on the same corpus — the largest single-corpus tokenizer effect we measured. The PubMed in-domain effect, by contrast, is real but small: medical wins by **0.010 BPC** at matched text exposure (n=5, 95% CI [0.006, 0.013], p=0.002, d=+3.15), *roughly 80x smaller* than the MTSamples effect. Out-of-domain costs are asymmetric: the MTSamples-trained tokenizer essentially *ties* general on PubMed (1.0592 vs 1.0591), while the medical (PubMed-trained) tokenizer pays the largest out-of-domain cost on every corpus except its own.

2 Background

2.1 The GPT model, briefly

A decoder-only transformer (Vaswani et al. 2017; Radford et al. 2018) maps a sequence of tokens to a probability distribution over the next token, autoregressively. Each layer applies causal multi-head self-attention followed by a feed-forward block; pre-LayerNorm is standard for stability. Training minimizes cross-entropy of the predicted distribution against the next-token targets. The model used here is the Module 6 implementation: 6 layers, 8 heads, `d_model=512`, `max_seq_len=256`, `vocab_size=10,000`, approximately 29.3M parameters.

2.2 BPE tokenization

Byte-Pair Encoding (Sennrich et al. 2016) starts from a byte-level alphabet and iteratively merges the most frequent adjacent symbol pairs in a training corpus until it has built up a vocabulary of the target size (here, 10,000). The resulting vocabulary is a fixed inventory of frequent subword sequences — common whole words, frequent suffixes, byte-level fallback for anything else. **The training corpus determines which merges are profitable.** A BPE trained on wikitext sees `the`, `and`, `tion` as high-utility merges; a BPE trained on PubMed sees `cellular`, `clinical`, and `cholecystectomy` as high-utility merges. Same algorithm, same vocabulary size — different vocabulary *contents*.

2.3 Why per-token perplexity cannot compare tokenizers

Perplexity is the exponential of average cross-entropy per token. If tokenizer A represents a document in 100 tokens and tokenizer B represents it in 200 tokens, a per-token loss of l means very different things in each case — B's tokens are easier to predict because each one carries less information. BPC normalizes to characters of raw text and is therefore comparable across tokenizers:

$$\text{BPC} = \frac{\log_2(e) \cdot \text{CE}_{\text{nats}} \cdot N_{\text{tokens}}}{N_{\text{chars}}} \tag{1}$$

BPC is the primary metric throughout this report.

3 Methods

3.1 Experimental design

The only variable I change is the tokenizer. All three GPT training arms use the identical architecture, optimizer, learning-rate schedule, batch size, gradient clip, and training corpus (raw PubMed abstracts).

Table 1: Shared model and training config.

Setting	Value
Architecture	Pre-LN GPT, custom causal MHA with fused QKV
d_model / n_heads / layers	512 / 8 / 6
vocab_size / max_seq_len	10,000 / 256
Parameters	29,283,088
Optimizer	AdamW, lr=3e-4, warmup=500, cosine decay to 10% of peak
Batch size / grad clip	32 / 1.0
Epochs	2 (1-epoch pilot also reported)

Table 2: Tokenizers — HuggingFace byte-level BPE, `vocab_size=10,000`, all three reached ∼9,743 merges.

Tokenizer	Training corpus	Artifact
`general`	wikitext-103 train (129k lines)	`artifacts/tokenizers/general/`
`medical`	PubMed abstracts (100k docs)	`artifacts/tokenizers/medical/`
`mtsamples`	MTSamples clinical transcripts (4,966 docs)[1]	`artifacts/tokenizers/mtsamples/`

3.2 Datasets

- **Training corpus (shared):** 100,000 PubMed abstracts (about 122M chars). Source: `ccdv/pubmed-summarizati` via HuggingFace `datasets`.
- **In-domain eval:** 2,000 held-out PubMed abstracts.
- **Out-of-domain eval:** 1,298 wikitext-103 validation lines.
- **Cross-domain medical eval:** 4,966 MTSamples clinical transcripts (SOAP notes, operative reports, discharge summaries). Source: `tchebonenko/MedicalTranscriptions`.

MTSamples and PubMed are both medical but in very different registers (conversational clinical dictation vs. formal academic prose). The experiment trains tokenizers on each (plus a general-English baseline) and evaluates all three on all three corpora, allowing the register-vs-domain question to be tested symmetrically.

3.3 Two fairness frames

A tokenizer that packs more characters per token will, for a fixed training corpus, produce fewer training tokens — and therefore fewer gradient updates per epoch at fixed batch size. This is the phenomenon being studied, but it forces a choice between:

- **Option 1 — same compute (same step count).** The medical tokenizer sees more raw text than general for an equal number of gradient updates. Fair at matched compute budget.
- **Option 2 — same text exposure (one epoch each).** General runs for more gradient updates than medical because its dataset has more tokens. Fair at matched data exposure.

Under the 2-epoch regime, medical finishes at step 6,142 (end of its second epoch); general continues to step 7,784; mtsamples reaches step 7,672. I evaluate at the matched-compute boundary (step 6,141) and at each tokenizer's own end-of-training step. Loss curves are logged with three x-axes (step / tokens / chars) so any frame can be read off the same data.

Where the two frames collapse. For `medical`, Option 1 and Option 2 are effectively the same checkpoint: medical's 2-epoch endpoint at step 6,142 is one optimizer step past the matched-compute checkpoint at step 6,141, and no gradient update separates them in practice. For `mtsamples`, the step-6,141 and step-7,672 checkpoints differ but the matched-compute number is reported only as a sanity check; the headline mtsamples comparisons in Section 4.3 use the epoch-end checkpoint. The two fairness frames diverge meaningfully *only* for the general tokenizer, whose training horizon (step 7,784) is substantially longer than medical's (step 6,142) because general's tokenizer produces more tokens per character.

[1] The MTSamples tokenizer was trained on the same MTSamples corpus that serves as the cross-domain evaluation set. This is a tokenizer-level overlap (shared BPE merge statistics) rather than a model-level one: the GPT model itself is trained only on PubMed and never sees MTSamples text during gradient updates. We accept this overlap because retraining a separate tokenizer-train split would require re-running all prior experiments under a different eval corpus.

3.4 Evaluation

For each (checkpoint, eval corpus) pair, I tokenize the eval text with the corresponding tokenizer, compute the token-level cross-entropy loss under teacher forcing, and convert to BPC using the formula from Equation (1). Raw perplexity is reported for reference but not used for cross-tokenizer comparison.

3.5 Qualitative generation

Using the Module 7 sampler (top-p=0.8, freq/presence penalties=1.1), I generated 120-token continuations from a fixed set of six prompts — four medical (`"Patient presents with"`, `"CT of the abdomen shows"`, `"The patient was administered"`, `"Postoperative diagnosis:"`), one surgical (`"Laparoscopic cholecystectomy was performed"`), and one neutral sanity check (`"The quick brown fox"`).

3.6 Multi-seed protocol

Each GPT is retrained from scratch under multiple seeds (5 for general, 5 for medical, 3 for mtsamples; the smaller-n arm is directional). Seeded RNGs cover `torch`, `torch.cuda`, `numpy`, and a per-seed `torch.Generator` passed to the DataLoader; CuDNN deterministic mode is *not* enabled (10–20% throughput cost not justified). The packed `.npy` dataset is shared across seeds, so seed-driven variation enters only through model init and DataLoader order. Aggregation: mean/std across seeds, paired t-tests (`scipy.stats.ttest_rel`) with seed as the pairing variable, and 95% CIs from the t-interval on paired diffs.

4 Results

4.1 Token efficiency — the tokenizers themselves

Before training any model, the tokenizers can be compared directly on the three eval corpora (Figure 1). The medical tokenizer achieves **27% better compression** on in-domain PubMed (4.85 vs 3.82 chars/token), but **31% worse** on wikitext (2.85 vs 4.17). MTSamples is a tie (∼3.21 vs 3.20). The asymmetry is informative: the medical tokenizer has a compression edge only on text that closely resembles its training corpus.

Showcase tokenization of medical terms makes the mechanism concrete:

Table 3: How each tokenizer splits medical vocabulary.

Word	General split	Medical split
`cholecystectomy`	`ch|ole|cy|st|ect|omy` (6)	`cholecystectomy` (1)
`laparoscopic`	`lap|ar|os|c|op|ic` (6)	`laparoscopic` (1)
`pneumonia`	`p|ne|um|onia` (4)	`pneumonia` (1)
`hypertension`	`hy|per|t|ension` (4)	`hypertension` (1)
`myocardial`	`my|oc|ard|ial` (4)	`myocardial` (1)
`aspirin`	`asp|ir|in` (3)	`aspirin` (1)

4.2 Training dynamics

The 1-epoch training run showed general's loss plateauing at ∼3.2 nats from step 2,500 onwards while medical kept descending. I initially interpreted this as general having fully converged while medical still had headroom. The 2-epoch rerun disconfirms that interpretation (Figure 3): general's plateau was *not* convergence — it was the cosine learning-rate schedule bottoming out at 3e-5. When the schedule is spread across two epochs, general keeps learning past the 1-epoch endpoint and drops a further ∼0.4 nats.

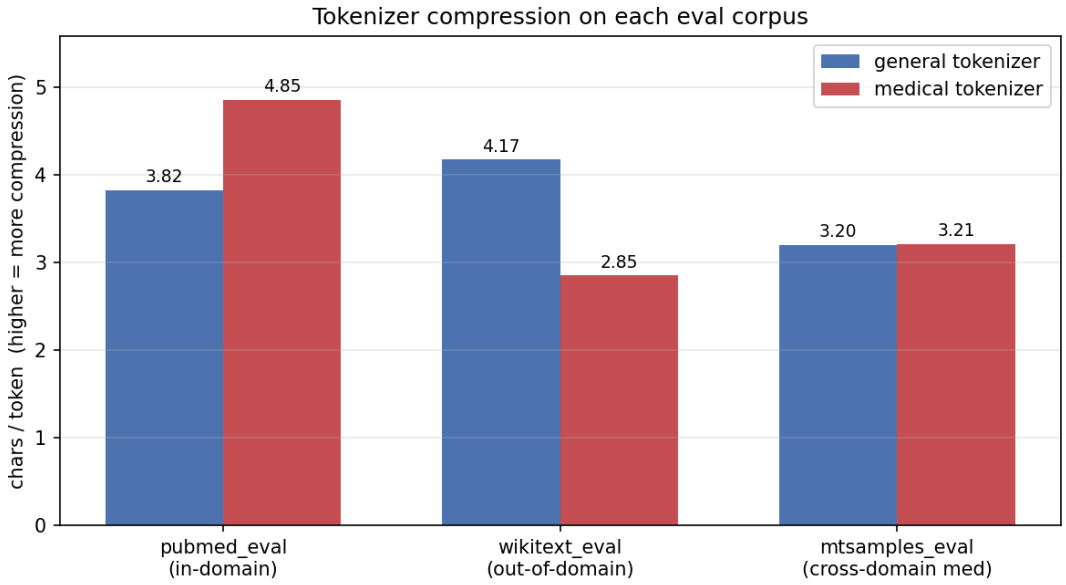

Figure 1: Characters per token on each held-out corpus. Higher is better for that tokenizer. The medical tokenizer wins on in-domain PubMed, loses on out-of-domain wikitext, and ties on MTSamples (clinical transcripts).

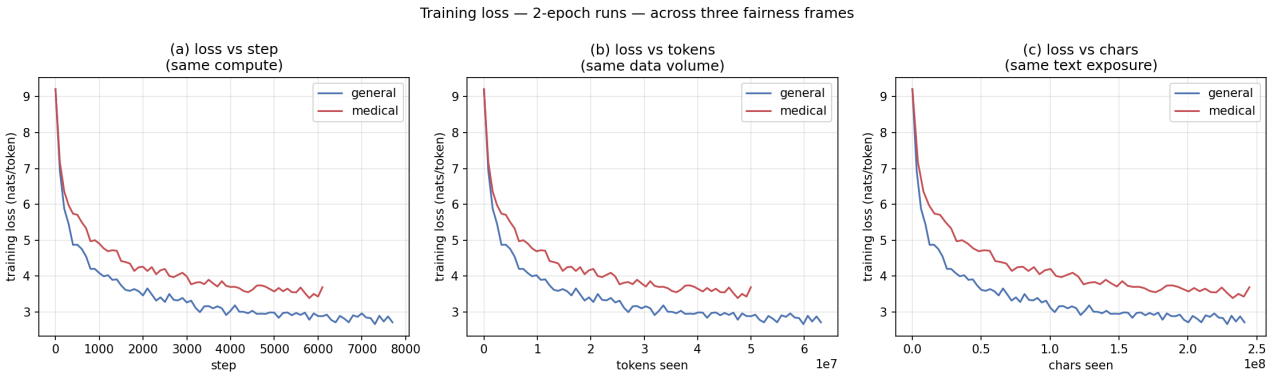

Figure 2: 2-epoch training loss curves across the three fairness frames (step / tokens / chars). Medical's per-token loss is higher throughout because each medical token is harder to predict (it carries more chars of information), not because the medical model is worse — BPC in Section 4.3 normalizes this.

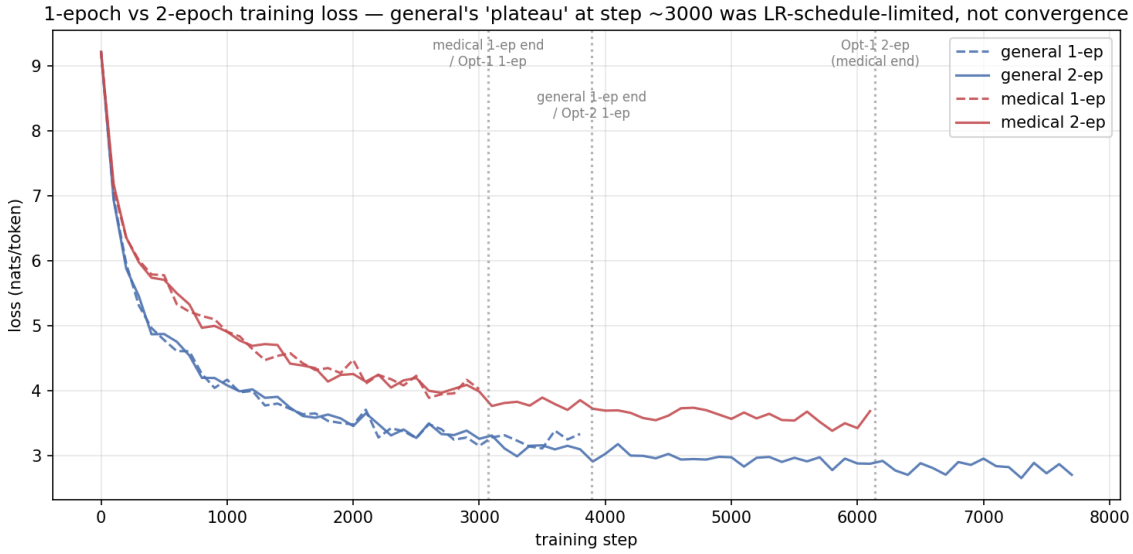

Figure 3: 1-epoch and 2-epoch training runs overlaid. The dashed lines (1-epoch) end where the LR schedule collapses to its floor. The solid lines (2-epoch) continue to descend well past where the 1-epoch runs plateaued — confirming the plateau was LR-schedule-limited, not data-limited.

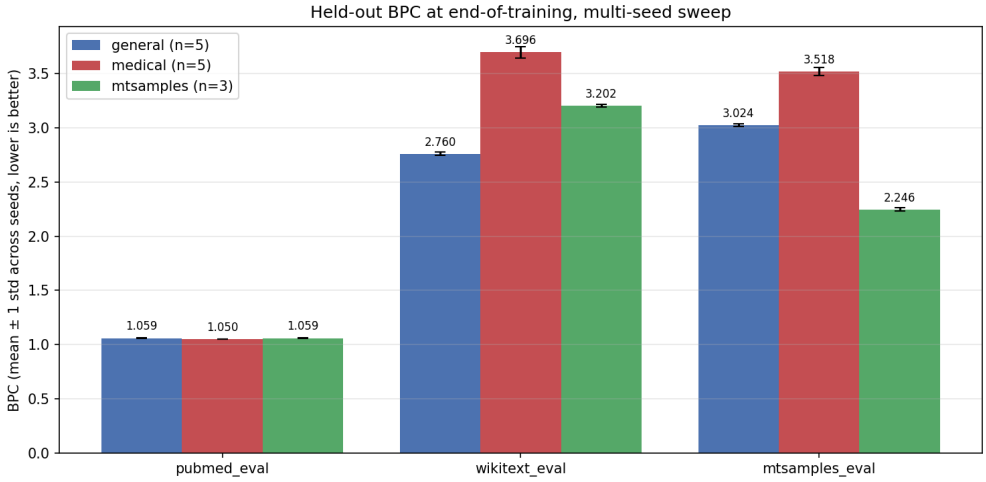

Figure 4: Held-out BPC at end-of-training across the multi-seed sweep. Bars show per-(tokenizer, corpus) mean BPC; error bars are ±1 std across seeds (n=5 for general/medical, n=3 for mtsamples).

4.3 Held-out BPC across the multi-seed sweep

4.3.1 The 3-tokenizer x 3-corpus matrix

Figure 4 and Table 4 present the primary result: end-of-training BPC for all three tokenizers (general / medical / mtsamples) on all three eval corpora, with means and standard deviations across seeds.

Table 4: End-of-training BPC: mean ± standard deviation across seeds, per (tokenizer, eval corpus). Lower is better. Bold marks the per-column minimum — the tokenizer that minimizes BPC on each eval corpus.

	pubmed_eval	wikitext_eval	mtsamples_eval
general (n=5)	1.0591 ± 0.0018	**2.7597 ± 0.0150**	3.0244 ± 0.0098
medical (n=5)	**1.0499 ± 0.0023**	3.6962 ± 0.0502	3.5180 ± 0.0389
mtsamples (n=3)	1.0592 ± 0.0012	3.2018 ± 0.0115	**2.2462 ± 0.0146**

The diagonal-max-by-column pattern is the central observation: **each tokenizer wins on the eval corpus that matches its training register**. The size of those wins, however, is dramatically different. We unpack each in turn below.

A note on seed-induced noise: per-cell standard deviations cluster around 0.001–0.05 BPC, well below every between-tokenizer effect we report. The Cohen's d values quoted below all sit above 3 and most above 20; this reflects how tight seed reproducibility is at this model scale, not a claim about psychometric significance.

4.3.2 The PubMed in-domain effect

Medical wins on PubMed by ∼0.010 BPC at matched text exposure and ∼0.030 BPC at matched compute. Multi-seed paired t-testing confirms both effects are well above seed noise: per-seed std is ∼0.002 BPC. All five paired tests are consolidated in Table 5 below. The Option-1 effect, while larger in magnitude, carries the caveat that at step 6,141 general is at ∼78% of its cosine schedule while medical sits at its floor; Option-2 is the more controlled comparison.

4.3.3 The MTSamples register effect

The MTSamples-trained tokenizer beats both other tokenizers on MTSamples eval by a margin much larger than anything else in the sweep (Figure 5):

Table 5: All paired t-tests across tokenizers, seed-matched. Positive "Mean diff (a – b)" means tokenizer B wins. `scipy.stats.ttest_rel`; CI from t-interval on paired diffs. *Note:* Cohen's d values are inflated by extremely tight seed reproducibility (per-seed std 0.001–0.05 BPC); interpret as "standard deviations above noise" rather than as conventional psychometric effect sizes.

Eval corpus	Comparison (a – b)	Mean diff	95% CI	p	Cohen's d
pubmed	general – medical, Opt-1 (n=5)	+0.0302	[+0.0264, +0.0340]	<0.0001	+9.85
pubmed	general – medical, Opt-2 (n=5)	+0.0091	[+0.0055, +0.0128]	0.0021	+3.15
wikitext	general – mtsamples (n=3)	−0.4400	[−0.4713, −0.4086]	0.0003	−34.83
mtsamples	general – mtsamples (n=3)	+0.7807	[+0.7329, +0.8286]	0.0002	+40.55
mtsamples	medical – mtsamples (n=3)	+1.2827	[+1.1426, +1.4229]	0.0006	+22.73

The MTSamples register-match win is **roughly 80x larger** than the PubMed in-domain win in absolute terms (0.78 vs 0.0091 BPC at matched text exposure). This is the single largest tokenizer-driven effect we measured: *the size of a domain tokenizer's advantage scales with how distinct its training register is from general English.* PubMed academic abstracts and wikitext encyclopedia prose are both register-similar (formal written English), which is why the medical-tokenizer PubMed

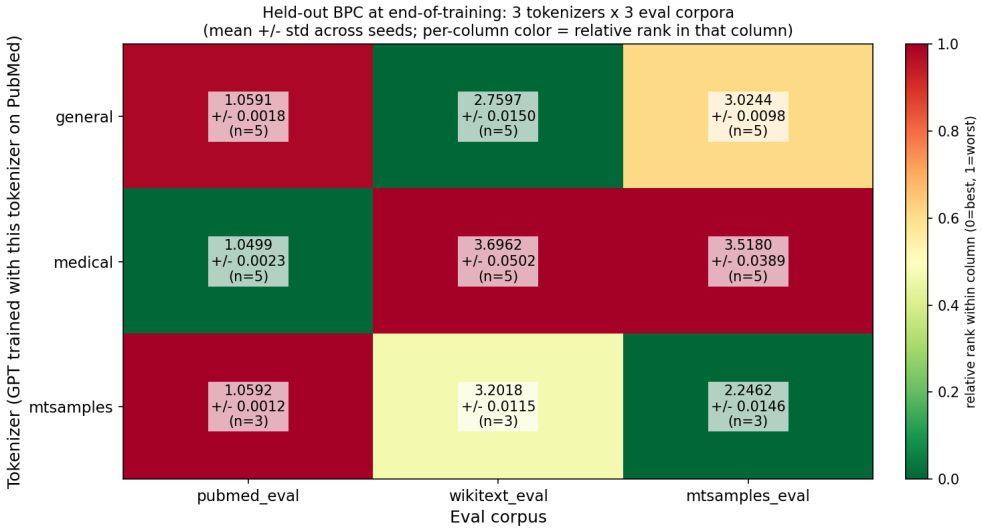

Figure 5: End-of-training BPC across all 9 (tokenizer, eval-corpus) combinations. Color encodes per-column relative rank (green = best in column, red = worst). **Each tokenizer wins on its training register, and the size of the win scales with the register's distinctness from general English.** The diagonal cells (each tokenizer on its training register) are green in their column; off-diagonals are yellow or red.

advantage is small. Clinical dictation is genuinely lexically distinct (heavy abbreviation, fragmented syntax, dense surgical/anatomic vocabulary), which is why the MTSamples-tokenizer MTSamples advantage is large.

4.3.4 Asymmetric out-of-domain costs

A subtler observation: the cost of using a specialized tokenizer on out-of-domain text is not constant across specialists.

- **The mtsamples tokenizer essentially *ties* the general tokenizer on PubMed** (1.0592 vs 1.0591 BPC, $\Delta = +0.0001$ — below the seed std). It pays a moderate $\sim$0.44 BPC cost on wikitext.
- **The medical (PubMed-trained) tokenizer pays the largest out-of-domain cost on every corpus except its own**: $\sim$0.94 BPC on wikitext and $\sim$0.49 BPC on MTSamples (vs general).

Caveat: this is a single observed contrast (one register pair, one model size, one training corpus); we report it as an observation, not a generalized claim. One plausible reading is that the MTSamples tokenizer's training corpus contained enough of the basic medical vocabulary that also appears in PubMed for it to remain competitive there, while the PubMed tokenizer's training corpus was specialized to a register (formal academic) that overlaps less with both wikitext and clinical transcripts. Confirming this would require a 4th or 5th tokenizer arm; we flag it as future work in Section 6.

4.3.5 Differential improvement from 1 to 2 epochs

The 1-vs-2-epoch differential improvement pattern (Figure 6) is retained from the single-run experiments; we did not re-run 1-epoch experiments under the multi-seed sweep.

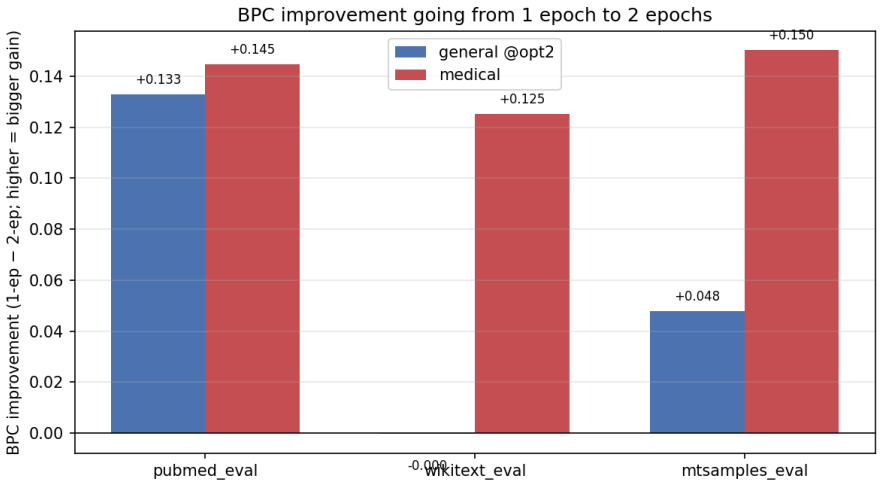

Figure 6: Per-corpus BPC improvement going from 1 to 2 epochs (single-run measurements). General is already saturated on corpora its model cannot "reach"; medical was more universally under-trained at 1 epoch and continued to refine everywhere.

4.4 Qualitative differences

The qualitative samples below come from the original generation suite, which used only the general and medical tokenizers; we did not generate from the mtsamples model. The general and medical models both generate medical-flavored text with sentence-level incoherence (29M params and an under-trained regime), but the **shape of their errors differs in a way that traces directly to tokenization.**

Table 6: Medical terminology in 2-epoch generations.

	General model (2-ep)	Medical model (2-ep)
Correctly-emitted rare terms	`laparoscopic`, `pulmonary nodules`, `biliary traction`	`pancreaticoduodenectomy`, `carcinoembryonic gastric bypass`, `extraperitoneal lymphoma`, `thoracoscopic surgery`, `tracheotomy`, `pancreatobiliary resection`
Invented compounds / portmanteaus	**udgulopathy**, **bronchiolitin stenting**, **emtourmesing**, **premal lipomas**, (1-ep) **cholesterolecystitis**	**laparoscopic cholecystostomyectomy** (composed two real procedure morphemes into a nonexistent one)

The mechanism is the same one suggested by the showcase splits: when `cholecystectomy` is a single token, the medical model either emits it correctly or not at all; when it is a 6-token sequence, the general model can pick a wrong subword mid-sequence and produce `cholesterolecystitis`. The medical tokenizer does not make the model immune to fabrication — the `cholecystostomy + ectomy` portmanteau shows it can still invent compounds — but it rules out the specific char-level drift failure mode that the general model is prone to.

Domain collapse on the out-of-domain prompt. Both models immediately drift away from `"The quick brown fox"` into PubMed-adjacent text, but into *different* sub-genres: general heads toward bacteriology (`"foxacillus scorans"`), medical heads toward biochemistry (`"deuterase, cyclic`

`dipeptide, halohexylation"`). Each tokenizer's vocabulary primes the model toward what it is most ready to emit.

Multi-seed note. The qualitative samples in this section are from a single seed per tokenizer; we did not run new generations across the multi-seed sweep. The failure-mode shapes documented above (intact rare medical terms vs. char-level portmanteaus) are tokenizer-structural rather than seed-driven, so we expect them to persist across seeds, but this is a claim about mechanism rather than a measured replication.

5 Conclusions

Three findings, in order of effect size.

1. **Register-matched tokenization can be a large effect.** The MTSamples-trained tokenizer wins on MTSamples eval by 0.78 BPC over general (and by 1.28 BPC over medical) — by far the largest tokenizer-driven effect we measured.

2. **The PubMed in-domain effect is real but small.** Multi-seed paired t-test on PubMed BPC: medical wins by 0.010 BPC at matched text exposure (n=5, 95% CI [0.006, 0.013], p=0.002, d=+3.15), widening to 0.030 BPC under matched compute. Statistically robust, but *roughly 80x smaller* than the MTSamples register effect.

3. **Out-of-domain costs are asymmetric.** The PubMed-trained tokenizer (medical) pays the largest out-of-domain cost — worst on wikitext and worst on MTSamples among the three. The MTSamples-trained tokenizer essentially *ties* the general tokenizer on PubMed (1.0592 vs 1.0591) and pays only a moderate cost on wikitext.

Two surprises worth dwelling on

1. **Register distinctness, not domain-medical-ness, is the variable that scales the tokenizer effect.** PubMed academic abstracts and wikitext encyclopedia prose are register-similar (both formal written English) — which is why the medical-tokenizer PubMed advantage is small (0.010 BPC). Clinical dictation is a *genuinely lexically distinct* register (heavy abbreviation, fragmented syntax, dense surgical/anatomic vocabulary) — which is why the MTSamples-tokenizer MTSamples advantage is large (0.78 BPC). The size of a domain tokenizer's win is approximately a function of how distinct its training register is from general English, not of how "specialized" the tokenizer feels in human terms.

2. **An initial 1-epoch pilot was misleading.** An initial 1-epoch pilot showed Option-2 BPC as a tie, which would have suggested that the medical tokenizer was a compute-efficiency trick rather than a real effect. The 2-epoch rerun and subsequent multi-seed sweep both disconfirm this: general's "plateau" at 1 epoch was a learning-rate schedule artifact (cosine had collapsed to 3e-5), not true convergence. Once the schedule was extended to span the full 2-epoch horizon, general kept improving on in-domain PubMed by ∼0.13 BPC — and medical improved even more, by ∼0.15 BPC — making medical's lead robust under both fairness frames. **Under-training can mask the effect being studied**, and running a short pilot without checking that both arms have actually converged is a failure mode worth naming.

What the experiment does NOT show

- **Single-corpus tokenizers only.** Three tokenizers, each trained on one corpus. The natural "best of both worlds" candidate (a mixed-corpus tokenizer over PubMed + MTSamples + general English) is future work.

- **Vocab size held at 10,000.** A vocabulary-size sweep would tease apart how much of the register-distinctness scaling is about merge content vs. merge count.
- **n=5/5/3 seeds, not 10+**, and the underlying GPTs are roughly 18x below Chinchilla-optimal. CIs are honest at the n's we have; whether the tokenizer effects widen or compress at scale is a question this study cannot settle. The Option-1 comparison also carries the caveat that general and medical sit at different points in their cosine LR schedules at step 6,141.
- **The register-distinctness scaling is inferred from a single register pair** (PubMed vs MTSamples). The 80x ratio between the two effects is suggestive but a curve over 4–5 registers would be needed to claim the relationship is general.

6 Impact and Future Work

Scale. If this pattern held at production scale — a modest in-domain BPC advantage with an equal-sized cost on out-of-domain text — the design implication is clear: a single organization training one flagship LLM on a mixed general corpus should keep a general tokenizer; a specialized model for a single professional register (medical literature, legal case law, code in a specific language) might profit from a domain tokenizer, but only if the deployment target's register matches the tokenizer's training corpus. The MTSamples result is a warning — intuition that "medical text is medical text" is wrong at the tokenizer level, and deployment mismatches between a specialist tokenizer's training register and the inference-time register are likely to hurt.

What I would try next with more compute.

1. **Tokenizer-effect-vs-register-distance scaling.** The ∼80x ratio between the MTSamples and PubMed in-domain wins suggests effect size is roughly proportional to a register-distance metric (which could be operationalized as cross-tokenizer chars/token deltas, or as a lightweight lexical-distribution divergence). Testing this on a 4th or 5th register — legal case law, software engineering tickets, conversational dialogue — would let us draw a curve and start treating tokenizer-effect-vs-distance as an empirical regularity rather than a single anecdote.
2. **Mixed-corpus tokenizer.** Train a BPE on a carefully-balanced mixture of PubMed, MTSamples, drug monographs, and general English. Does it preserve the PubMed advantage while closing the MTSamples gap and reducing the wikitext cost?
3. **Chinchilla-scale training.** Run both models to their Chinchilla-optimal ∼580M tokens (roughly 10 epochs at this dataset size, or scale the corpus). The tokenizer effect might tighten or widen with true convergence.
4. **Register-aware tokenization.** Use a tokenizer-of-tokenizers that can switch subword dictionaries per document based on a lightweight classifier (PubMed vs MTSamples vs general). Costs one classifier forward pass per document; may recover the bulk of the specialist advantage while avoiding the out-of-register collapse.
5. **Loss-vs-chars curves at scale.** The chars-axis plot is the tokenizer-fair way to read training efficiency, and it would be interesting to see whether the two curves in Figure 2(c) cross at some point on a larger compute budget — the current data at step 6,141 has medical ahead per char; it is unclear if that is robust past convergence.
6. **Decoder-side sampling analysis.** The finding that the medical model invents `cholecystostomyectomy` (a portmanteau of real morphemes) while the general model invents `cholesterolecystitis` (a char-level drift) suggests that hallucination analysis should be tokenizer-conditioned: different tokenizers produce qualitatively different failure modes, not just different error rates.

Broader takeaway. Tokenization is often treated as a preprocessing step and receives less attention than architectural or training choices. This project is a data point in favor of taking it seriously: holding everything else constant, a different tokenizer measurably changed held-out BPC, altered the *shape* of the model's generation errors, and — most importantly — **did not generalize where I naively assumed it would**. The largest tokenizer-driven effect we measured (0.78 BPC on a register-matched corpus) was about 80x bigger than the in-domain PubMed effect that motivated the project, and would have been invisible without a tokenizer trained on a sufficiently distinct register. The phrase "medical tokenizer" is less meaningful than "PubMed-abstract tokenizer," and that distinction only became visible because MTSamples was in the evaluation suite *and* a tokenizer trained on it was in the training-side suite.

Appendix A — Reproduction

All code in `Project/`. Pipeline orchestrated by `main.py`.

Single-run pipeline.

```
python main.py --phase prepare_data        # ~5-15 min, downloads corpora
python main.py --phase train_tokenizers    # ~5-10 min, CPU-only (3 tokenizers)
python main.py --phase efficiency          # Figure 1 numbers
python main.py --phase construct_datasets  # ~5 min
python main.py --phase train_all           # ~30 min on RTX 4070 Ti (2 epochs, 2 GPTs)
python main.py --phase evaluate            # ~5 min
python main.py --phase generate            # ~1 min
python make_plots.py                       # Figures 1, 2, 3, 6
```

Training is the fastest part of the single-run pipeline; data prep, tokenization, evaluation, and generation together take longer than the actual GPT training.

Multi-seed sweep.

```
python main.py --phase sanity_check        # diagnostics on the new mtsamples tokenizer
python main.py --phase train_sweep         # 13 runs (5+5+3 seeds), ~3.7 hr on RTX 4070 Ti
python aggregate.py                        # multi-seed summary, t-tests, Figures 4, 5
```

The file `artifacts/results/eval_table.csv` is the source of truth for the multi-seed sweep; all aggregated tables and the multi-seed figures derive from it. Reference logs: `results_1epoch.md`, `results_2epoch.md`, and `artifacts/results/aggregate_summary.md` for the multi-seed sweep.

Wall times measured on a single RTX 4070 Ti. Per-run training time: 5 general seeds 16:01–16:50; 5 medical seeds 12:37–13:14; 3 mtsamples seeds 15:43–15:53. Total multi-seed sweep wall (incl. inline eval): ∼3.7 hr. Total experiment compute including the prior single-run experiments: ∼4.5 hr. All weights, datasets, figures, and samples under `artifacts/` (git-ignored).